

Calculadora {

```
<Por="Pablo Bielsa y  
        Nicolae Poenaru"/>
```

}



Nuestro proyecto {

Se encarga de resolver operaciones básicas como suma, resta, multiplicación y división, además de funciones más avanzadas como el cálculo de porcentajes y la obtención del valor máximo, todo estructurado de forma limpia y modular.

```
2
3 public class Calculadora {
4
5     public double sumar(double a, double b) {
6         return a + b;
7     }
8
9     public double restar(double a, double b) {
10        return a - b;
11    }
12
13    public double multiplicar(double a, double b) {
14        return a * b;
15    }
16
17    public double dividir(double a, double b) {
18        if (b == 0) {
19            throw new ArithmeticException("No se puede dividir por cero");
20        }
21        return a / b;
22    }
23
24    public double maximo(double a, double b) {
25        if (a >= b) return a;
26        return b;
27    }
28
29    public double porcentaje(double valor, double total) {
30        if (total == 0) {
31            return 0.0;
32        }
33        return (valor / total) * 100;
34    }
35 }
```

}

Planificación de Pruebas {

Objetivos:

Asegurar la integridad matemática y validar la gestión de excepciones según la norma ISO 29119.

Alcance:

Pruebas dinámicas de caja negra, caja blanca y valores limite sobre todos los métodos de la clase Calculadora.

Criterios:

Entrada: código que compile bien y reciba los datos correctamente.
Salida: Que haya un reporte técnico sin fallos críticos.

}

Estructura {

ID	Operacion	Entradas	Esperado	Técnica
TC-001	SUMA SIMPLE	5, 3	8	CAJA NEGRA
TC-004	RESTA SIMPLE	10, 4	6	CAJA NEGRA
TC-006	MULTIPLICACION POR CERO	6, 0	0	VALORES LIMITE
TC-009	DIVISION POR CERO	10, 0	0	VALORES LIMITE
TC-010	MÁXIMO CUANDO A > B	8, 3	8	CAJA BLANCA
TC-001	PORCENTAJE NORMAL	25, 200	12.5	CAJA NEGRA

}

Implementación con JUnit 5 {

```
11 @DisplayName("Pruebas Unitarias Avanzadas - ISO/IEC 29119")
12 public class CalculadoraTest {
13     private Calculadora calc;
14     @BeforeEach
15     void setUp() {
16         calc = new Calculadora();
17     }
18     @Test
19     @DisplayName("Suma de dos números positivos comunes")
20     void testSumarPositivos() {
21         assertEquals(8.5, calc.sumar(5.0, 3.5), 0.001, "5.0 + 3.5 debería ser 8.5");
22     }
23     @Test
24     @DisplayName("Suma de dos números negativos")
25     void testSumarNegativos() {
26         assertEquals(-6.0, calc.sumar(-2.0, -4.0), 0.001);
27     }
28     @Test
29     @DisplayName("Suma con valor frontera (Elemento neutro cero)")
30     void testSumarCero() {
31         assertEquals(10.0, calc.sumar(10.0, 0.0), 0.001);
32     }
33     @Test
34     @DisplayName("Resta básica de dos números positivos")
35     void testRestarPositivos() {
36         assertEquals(6.0, calc.restar(10.0, 4.0), 0.001);
37     }
38     @Test
39     @DisplayName("Resta cuyo resultado produce un número negativo")
40     void testRestarResultadoNegativo() {
41         assertEquals(-3.0, calc.restar(2.0, 5.0), 0.001);
42     }
43     @Test
44     @DisplayName("Multiplicación por cero (Caso Límite)")
45     void testMultiplicarPorCero() {
46         assertEquals(0.0, calc.multiplicar(6.0, 0.0), 0.001);
47     }
48     @Test
49     @DisplayName("Multiplicación de dos números negativos (Regla de signos)")
50     void testMultiplicarNegativos() {
51         assertEquals(9.0, calc.multiplicar(-3.0, -3.0), 0.001);
52     }
53     @Test
54     @DisplayName("División exacta estándar")
55     void testDividirNormal() {
56         assertEquals(5.0, calc.dividir(10.0, 2.0), 0.001);
57     }
58 }
```

}

Resultados de ejecucion{

Cover

CalculadoraTest x

✓ Pruebas Unitarias Avanzadas · 93 ms

✓ Suma de dos números posi 40 ms

✓ Multiplicación por cero (Cas 1 ms

✓ Caja Blanca: Segundo parám 2 ms

✓ Suma de dos números negat 1 ms

✓ Resta básica de dos número 1 ms

✓ Porcentaje cuando el total e 1 ms

✓ Multiplicación de dos número 1 ms

> ✓ Cálculo de porcentaje sobr 40 ms

✓ Resta cuyo resultado produc 1 ms

✓ División exacta estándar 1 ms

✓ Caja Negra: Ambos valores son idé

✓ Dividir entre cero debe lanza 3 ms

✓ Suma con valor frontera (Elemento

✓ Caja Blanca: Primer parámet 1 ms

✓ 16 tests passed 16 tests total, 93 ms

C:\Users\Pablo\.jdk\ms-17.0.19\bin\java.exe ...

Process finished with exit code 0

}

Cobertura JaCoCo {

El análisis de cobertura arrojó un 100% de éxito tanto en instrucciones como en ramas lógicas de la clase Calculadora.java. Este porcentaje máximo certifica que la suite de 15 pruebas automatizadas consiguió ejecutar y auditar cada línea de código, camino condicional y bloque de excepción del programa, garantizando que no existe nada que se haya quedado fuera de la verificación técnica.

Coverage CalculadoraTest ×				
Element ^	Class, %	Method, %	Line, %	Branch, %
> cesur.daml.practica	100% (1/1)	100% (6/6)	100% (11/11)	100% (6/6)

}

Fallos / bugs {

- Bug 1: División por cero sin protección. Solución: Validación previa con lanzamiento de excepción.
- Bug 2: Desviación lógica en el método del valor máximo con números negativos. Solución: Corregir la lógica del código.



}

Resumen {

Resumen: Trabajar con Git desde el primer día y automatizar las pruebas a tiempo nos ha evitado muchos problemas de última hora. Este proceso nos ha permitido detectar y corregir los errores sobre la marcha, asegurando que entregamos una aplicación sólida, bien estructurada y completamente revisada.

Dificultades: Al principio de la fase de pruebas tuvimos un pequeño problema: no encontrábamos el JDK de Java para configurar Maven.

Una vez que instalamos IntelliJ y enlazamos correctamente el entorno de desarrollo, pudimos desbloquear el proyecto, ejecutar los tests con éxito y obtener los resultados finales que mostramos en esta sección.

docs: crear archivo README con la guía de usuario y tecnologías del proyecto

 bielsaa777 committed 2 days ago

fix: corregidos errores logicos en maximo, porcentaje y division por cero

 bielsaa777 committed 2 days ago

test: implementacion completa de los 15 casos de prueba JUnit 5

 bielsaa777 committed 2 days ago

design: diseno de la estructura de los casos de prueba JUnit

 bielsaa777 committed 2 days ago

feat: estructura inicial del proyecto y código fuente de Calculadora

 bielsaa777 committed 2 days ago

}

Gracias {

<Un besito a todos, guap@s 🥰>

}